UNDER REVIEW

RESCIENCE C

Replication / Geosciences

[Re] An open and extensible framework for spatially explicit land use change modelling: the lulcc R package

Sérgio Souza Costa[1, ID], Denilson da Silva Bezerra[1, ID], José Magno Pinheiro Alves[1, ID], Hissa Barbara Oliveira[1, ID], Mateus Dutra Vale[1, ID], and Samylli Kalei Silva Zhao[1, ID]

[1] LambdaGeo, Universidade Federal do Maranhão (UFMA), São Luís, Brazil

Edited by
(Editor)

Received
–

Published
–

DOI
–

1 Introduction

Moulds, Buytaert, and Mijic[1] presented lulcc, an R package for spatially explicit land use change modelling, with an explicit aim: to make land use change models open, so that their results can be reproduced and their code examined and extended. They demonstrated the package on the Plum Island Ecosystems data set, which ships with it, going through the stages of an inductive model: suitability from logistic regression, demand from interpolation between observed maps, spatial allocation by CLUE-S[2] and by an ordered procedure[3], and pattern validation by three-map comparison[4].
We replicate that application in Python: a replication in the sense of ReScience C[5], an independent implementation written from the article and run by us, rather than a reproduction with the original code. In the vocabulary of National Academies of Sciences, Engineering, and Medicine[6], which keeps the word replication for studies that collect new data, it is a study of the same data with new code. Both lulcc and the CLUE-S implementation we use were written from the published description of the algorithm[2], independently of each other. Ours comes from LuccME[7], a land use change framework developed at INPE on the TerraME modelling platform[8], which organises a model into demand, potential and allocation components and has supported land use and emission scenarios for the Brazilian Amazon[9] and for Brazil[10]. The original article itself lists LuccME among the implementations of CLUE-S. Allocation procedures are known to leave room for different readings, with consequences for the simulated maps[11]; for CLUE-S, the comparison therefore tests two independent readings of the same description, with lulcc's own demonstration as the reference.

2 Replication summary

Figure 1 summarises the replication. On the left is the reference: lulcc 1.0.4 running the article's own demonstration script, in a Docker image. On the right is the replication, which runs on DisSModel[12], a Python framework for spatially explicit dynamic models, and its land use package DisSLUCC[13]. In the centre are the four stages of the original model (Section 3), each implemented separately on both sides. The code of the replication is original, and its parts have three origins, shown in the right-hand boxes:

Copyright © 2026 S.S. Costa et al., released under a Creative Commons Attribution 4.0 International license.
Correspondence should be addressed to Sérgio Souza Costa (sergio.costa@ufma.br)
The authors have declared that no competing interests exist.
Code is available at https://github.com/LambdaGeo/rescience-lulcc..
Data is available at https://github.com/LambdaGeo/lulcc-goldens.

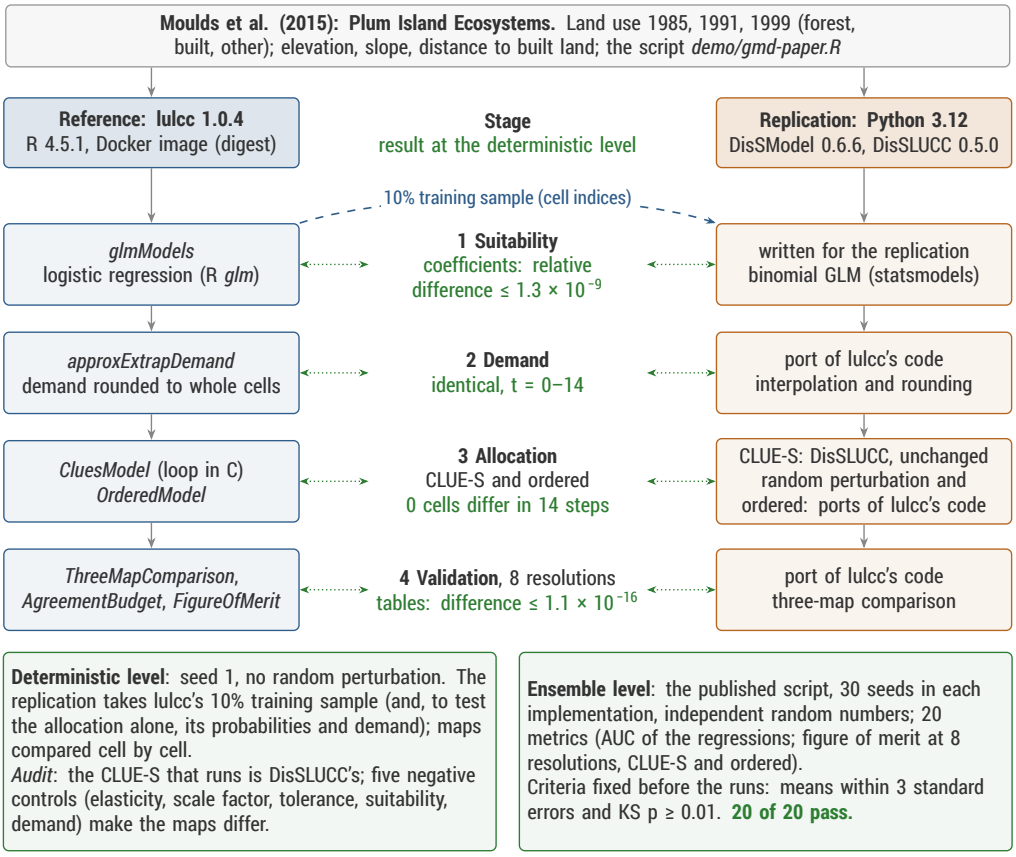


Figure 1. The replication and its comparison with lulcc. Left: the reference, lulcc's functions in the demonstration script. Right: the replication and the origin of each part. Centre: the four stages of the original model and, in green, the result of the deterministic comparison at each stage (Table 1). The dashed arrow, the cells drawn by R for the training sample of the regression, is the only input that the replication's own chain takes from the reference; the demand it computes itself. Bottom: the two levels of comparison (Section 7).

- *CLUE-S allocation*: the component of DisSLUCC 0.5.0, used unchanged at the deterministic level. It was ported, before and independently of this work, from the discrete allocation of LuccME, `AllocationDClueSLike`, and not from lulcc. In the three LuccME test labs that use it, DisSLUCC 0.5.0 reproduces the outputs of TerraME with the same number of iterations in every year and differences of at most 1.2×10^{-7}[14]; the same package runs the model of a national land use change workflow over Brazil[15].
- *Written for this replication, without reference to lulcc's code*: reading the data, the logistic regression (statsmodels) and its AUC, and the potential that passes suitability and elasticity to the CLUE-S allocation.
- *Ports of lulcc's code*: the demand rounding, the stratified training sample, the random perturbation of CLUE-S (a subclass of the DisSLUCC component that replaces its iteration loop), the ordered allocation, and the three-map comparison with its agreement budget and figure of merit. They follow the original article, and where it leaves a choice open they follow lulcc's procedure step by step, in Python (Section 4); their agreement with lulcc verifies the ports, not an independent reading.

The comparison has two levels, at the bottom of Fig. 1. At the *deterministic* level the randomness is removed from both sides, and the stages can be fed with the reference's

own inputs, so that a difference points at one stage; run as a whole chain, the replication takes from lulcc only the cells of the training sample, because R's random sample cannot be drawn again in Python. At the *ensemble* level both sides run the published script, with its randomness, using independent random numbers, and are compared in distribution. Two parts of the original application are not replicated: the regression trees and random forests of the paper's Fig. 5, since we did not reimplement R's rpart and randomForest, and the published curves of Figs. 7–8, whose values the paper does not give; we compare those figures with our own runs of lulcc instead, and the AUCs printed in Fig. 5 for all three model types with lulcc's own spread (Section 8.3).
Given the same inputs, the CLUE-S allocation of DisSLUCC reproduces every simulated map of lulcc cell by cell, and so do our ports of the ordered allocation and of the validation tables (to 10^{-16}). With independent random numbers, 30 runs of each implementation agree in distribution on all 20 metrics compared, under criteria recorded before the runs. Running the original demonstration script required one repair, unrelated to the model (Section 9).

3 The original model

The Plum Island Ecosystems data set, which lulcc distributes as derived from the study of Pontius and Parmentier[16], covers a site in north-east Massachusetts on a grid of 434×497 cells of about 100 m (99.92 m by 99.95 m), 113 563 of them inside the study area. It holds land use maps for 1985, 1991 and 1999 (forest, built, other), treated as time points 0, 6 and 14, and three explanatory variables: elevation, slope and distance to built land in 1985.

Suitability. One binary logistic regression per class is fitted on a stratified random sample of 10% of the cells of the 1985 map: built on the three variables, forest and other on elevation and slope. The paper also fits regression trees and random forests and compares the three model types by the area under the ROC curve (AUC) on the remaining cells (its Fig. 5), and evaluates the glm for built on the cells that became built between 1985 and 1991 (its Fig. 6).

Demand. The number of cells of each class at every time point from 0 to 14 is interpolated linearly between the three observed maps and rounded to integers that sum to the number of cells.

CLUE-S allocation. At each time step every cell receives, for each class, its suitability plus an elasticity (0.2 for all classes) if the cell already belongs to that class. Each cell is allocated to the class with the highest value plus a class-specific iteration term; the term is then increased by the difference between demand and allocated area, times a scale factor (10^{-6}), until the largest difference is at most 50 cells. At every iteration the suitability is perturbed by a uniform random number in $[-0.0002, 0.0002]$ (`jitter.f`).

Ordered allocation. Classes are visited in a fixed order (built, forest, other). A class with rising demand takes the most suitable cells not yet settled; a class with falling demand releases its least suitable cells. In the stochastic version a cell qualifies only if its suitability exceeds (or, when releasing, falls below) a uniform random number.

Validation. The initial observed map (1985), the final observed map (1999) and the simulated map for 1999 are compared at eight resolutions, aggregating 2×2 to 256×256 cells, through the three-dimensional contingency table of Pontius, Peethambaram, and Castella[4]. The paper reports the agreement budget and the figure of merit of the forest-to-built transition for both allocation methods (its Figs. 7 and 8).

4 Analysis of the original paper

The article states the model, its parameters and the code that produced its figures, which the package ships as a demonstration script (`demo/gmd-paper.R`). Four aspects are nonetheless not determined by the text.

Random numbers without a seed. Three steps draw random numbers: the training sample, the CLUE-S perturbation and the stochastic ordered allocation. The script sets no seed, so the numbers printed in the paper are one draw from a distribution and cannot be repeated exactly, not even with lulcc. This is why we generated the reference ourselves and compare at two levels (Section 7).

Choices left to the code. The text describes the following steps at the level of the algorithm. We followed lulcc's source code for them, so that a difference in the comparison would point at the algorithm rather than at a convention:

- demand rounding: rows rescaled to the number of cells, floored, and the remaining cells given to the classes with the largest remainders. LuccME rounds differently, adding each year's increment to the previous year's rounded demand, so this is a convention on which the two implementations differ;
- training sample: the class codes are numeric, so the stratification of `caret::createDataPartition`, which lulcc calls, groups them by quantile, which for this map gives two strata, {forest, built} and {other};
- ordered allocation: which cells are candidates, the temporary class for released cells, the limit of 1000 random draws, and ties kept in cell order;
- three-map comparison: the layout of the table, aggregation blocks anchored at the upper-left corner with incomplete edge blocks weighted by their number of valid cells, and empty denominators left out of the sums.

The CLUE-S allocation of DisSLUCC was not modified to match lulcc.

Behaviours visible only in the code. The random forest models receive a numeric 0/1 response and therefore run in regression mode. The CLUE-S loop has a second stopping condition based on the mean signed difference between demand and allocation; because the rounded demand sums to the number of cells, that mean is zero, and only the largest difference stops the iteration.

Iterations not reported. lulcc does not report the number of iterations of CLUE-S at each step, so only the map produced at each step can be compared, not the path of the iteration. Against LuccME the iteration counts are compared and agree.

Results shown only as figures. The AUCs are printed in the figures, but the agreement budgets and figures of merit are only drawn (Figs. 7–8). We compare the latter with our own runs of lulcc, not with the published curves.

5 Computational set-up

Data. The replication reads the Plum Island Ecosystems data set from GeoTIFF files exported from the R package (`data(pie)`), without any other change; the exported layers carry the original CRS (NAD83, Massachusetts Mainland, EPSG:26986).

Reference. We ran the demonstration script with lulcc 1.0.4[17] and archived the outputs[18]. Two references were produced:

- a *deterministic* one, with the randomness removed (seed 1 for the training sample, `jitter.f` $= 0$, ordered allocation not stochastic), exported stage by stage: training cells, regression coefficients, the suitability matrix, the demand table, all simulated maps and the three-map tables, with doubles written to 17 significant digits;
- an *ensemble* of 30 runs of the script as published, seeds 1 to 30.

The only change to the script is the repair described in Section 9.

Software. Reference: a Docker image based on rocker/geospatial 4.5.1 (pinned by digest, `sha256:6483198da9a6…`; full digest in the goldens' `Dockerfile`), with R 4.5.1 and packages from a dated snapshot of CRAN (1 October 2026): lulcc 1.0.4, raster 3.6-32, sp 2.2-0, caret 7.0-1, ROCR 1.0-12, randomForest 4.7-1.2, rpart 4.1.27. The lulcc 1.0.4 on CRAN was republished in November 2025, but its source package is the one built by the author in February 2019. Replication: Python 3.12, DisSModel 0.6.6, DisSLUCC 0.5.0, NumPy 2.5[19], SciPy 1.18[20], statsmodels 0.15[21], rasterio 1.5.

Hardware and time. The reference and the replication were run on one desktop computer (Dell OptiPlex 7050, Intel Core i7-7700T at 2.9 GHz, Linux Mint 22.2, based on Ubuntu 24.04). One run of CLUE-S over the 14 steps took on average 11 s in lulcc, whose inner loop is in C, and about 20 s in the replication; the ordered allocation took 3 s and 0.2 s. One complete run of the paper's experiment, including the regression trees and random forests of lulcc, took about 90 s in R; one run of the replication, about 30 s.

6 Implementation

Suitability is fitted with statsmodels (binomial GLM). The AUC is computed by the Mann–Whitney statistic, with ties counted as one half. CLUE-S is the allocation component of DisSLUCC (`AllocationDClueSLike`) on a DisSModel raster backend; the simulation starts at time 1, as lulcc does, and the suitability with elasticity is supplied to it in double precision. The random perturbation is added by a subclass of the DisSLUCC component that replaces its iteration loop and draws a new uniform number for every cell and class at every iteration; a test in the repository checks that, with the perturbation set to zero, it gives the same result as the published component. The ordered allocation and the three-map comparison are separate modules, operating on the valid cells in the raster's row-major order, which is also lulcc's cell order.

Why the maps can be identical. Two independently written implementations of an iterative allocation give the same map only if they agree on everything that decides a cell. lulcc's C loop and DisSLUCC's component do. The score of a class in a cell is its suitability, plus the elasticity if the cell belonged to that class at the start of the step, plus a class term that starts at zero at every step. Each cell takes the class with the highest score, the first class in order on a tie. After each pass the class term grows by the difference between demand and allocated cells times the scale factor, and the loop stops when the largest absolute difference is at most the tolerance (lulcc's second condition, on the mean difference, is inactive; Section 4). The parameters of DisSLUCC were translated from lulcc's, not tuned: the scale factor as `factor_iteration`, the tolerance as `max_difference`, a cell area of 1 so that demand is counted in cells, and a transition matrix of ones, the rule matrix of the demonstration script. The two differ only where this application never goes: at the iteration limit DisSLUCC stops with an error,

lulcc returns the last map with a warning, and so does the perturbation subclass of the ensemble; the limit was never reached.

7 Comparison and acceptance criteria

Criteria were recorded in the code repository before either ensemble was run; the commit that adds them precedes those with results.

Deterministic level. Each stage is fed with the reference's own inputs, so that a difference points at that stage only (centre of Fig. 1). Coefficients and suitability must agree within numerical precision; demand, allocated maps and validation tables must be identical or agree to rounding error.

Ensemble level. Our 30 runs (seeds 1 to 30 of NumPy's default generator) are compared with lulcc's 30 on 20 metrics: the AUC of the glm for each class and for the gain of built, and the figure of merit of forest to built for each allocation method at each of the eight resolutions. A metric passes if the means differ by at most three standard errors of the difference and a two-sample Kolmogorov–Smirnov test gives $p \geq 0.01$. The regression trees and random forests of the paper's Fig. 5 are outside the criteria: we did not reimplement R's rpart and randomForest, and other implementations would differ by construction.
The agreement budget of forest to built (the paper's Fig. 7) was added to the comparison after the ensembles had run, and is therefore outside the criteria: for its 64 values (four components, eight resolutions, two allocation methods) we report the same statistics for information only.

8 Results

8.1 Deterministic level

Table 1 lists the stages; the main results are also shown in the centre of Fig. 1. The CLUE-S allocation of DisSLUCC, given lulcc's suitability and demand, produced the same map as lulcc at each of the 14 time steps, cell by cell; it needed 51 iterations at the first step and 257 at the last. The ordered allocation also matched at every step, and the three-map tables and figures of merit agreed to rounding error. Running the whole chain from our own regression gave the same final map: the coefficients differ by about 10^{-9} in relative terms, which did not change any allocation. The same holds when the suitability is computed by DisSLUCC's own logistic potential, in single precision, from lulcc's coefficients, so the whole chain of DisSLUCC reproduces lulcc.
The agreement is not an artefact of the comparison. The CLUE-S that runs is the component of the installed package, called once per step; from the reference only the suitability and the demand are read, never a simulated map; lulcc's map changes at every step (12 733 cells at the first, 440 to 590 at the others), and ours matches the same step and not the neighbouring ones. The comparison also fails when it should: changing one thing at a time makes the final maps differ, by 4 555 cells for an elasticity of 0.25 instead of 0.2, 130 for a demand shifted by 100 cells, 45 for a tolerance of 10 cells instead of 50, 14 for noise of 10^{-4} added to the suitability and 6 for a scale factor doubled. These checks are in `scripts/audit.py`.
Figure 2 shows the observed maps and the 1999 maps simulated by CLUE-S in both implementations, which the original article does not show: it reports the simulations only through the agreement budget and the figure of merit. The replication's map comes from its whole chain, with its own regression and demand; only the training cells are

Stage	Result
Observed transitions 1985–1999 (paper, Sect. 3.2)	identical
Regression coefficients, same training cells	max. relative difference 1.3×10^{-9}
Suitability from lulcc's coefficients	max. difference 9×10^{-14}
Suitability from our coefficients	max. difference 2.8×10^{-10}
Demand, $t = 0, \ldots, 14$	identical
CLUE-S, 14 steps	0 cells differ at every step
Ordered allocation, 14 steps	0 cells differ at every step
Three-map tables, 8 resolutions, both methods	max. difference 1.1×10^{-16}
Figure of merit forest → built	max. difference 2.2×10^{-16}
Whole chain from our own regression	final map identical
DisSLUCC's own logistic potential (single precision)	0 cells differ at every step
Perturbation subclass with perturbation 0	0 cells differ at every step

Table 1. Deterministic level: each stage fed with the reference's inputs.

taken from the reference. The two simulated maps are the same map. Both look more clustered than the observed 1999 map, an impression we did not quantify; since the two maps are identical, it is a property of the model and its set-up, not of either implementation.

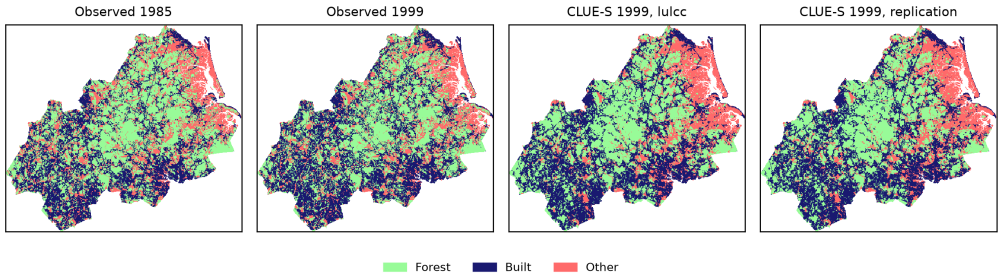


Figure 2. Land use observed in 1985 and 1999, and simulated for 1999 by CLUE-S without perturbation, by lulcc and by the replication (its own regression, demand and DisSLUCC's allocation; training cells of the reference). 0 cells differ between the two simulated maps. Colours as in Fig. 3 of the original article.

8.2 Ensemble level

All 20 metrics passed. Table 2 shows a selection and Figure 3 the figure of merit at every resolution; the spread between runs is the same in both implementations, including the larger spread of the stochastic ordered allocation. Figure 4 shows the agreement budget, which the criteria do not cover (Section 7): over its 64 values the means of the two implementations differ by at most 10^{-4} of the study area, at most 1.02 standard errors of the difference, and the smallest Kolmogorov–Smirnov p is 0.24. As in the original article, the change simulated correctly grows as the resolution coarsens, because errors of location within a block cancel.

The criteria detect a wrong model but not every wrong detail. With an elasticity of 0.25, ten runs of the replication fail seven of the eight figure-of-merit metrics of CLUE-S; without any perturbation they fail none. Most of the spread between runs comes from the training sample, and the perturbation changes only a few cells (19 in the final map of one run), so the ensemble level confirms the distribution of the results, not the implementation of the perturbation.

Metric	lulcc mean (sd)	replication mean (sd)	diff.	tol.	KS p
AUC glm, forest	0.6439 (0.0007)	0.6437 (0.0006)	−0.0003	0.0005	0.07
AUC glm, built	0.9382 (0.0003)	0.9382 (0.0003)	−0.0001	0.0002	0.81
AUC glm, other	0.7295 (0.0010)	0.7290 (0.0008)	−0.0004	0.0007	0.14
AUC glm, gain of built	0.6446 (0.0004)	0.6446 (0.0004)	0.0000	0.0003	0.81
FoM CLUE-S, factor 2	0.1168 (0.0017)	0.1167 (0.0019)	−0.0001	0.0014	0.81
FoM CLUE-S, factor 16	0.3766 (0.0058)	0.3756 (0.0062)	−0.0010	0.0046	0.81
FoM CLUE-S, factor 256	0.7684 (0.0052)	0.7688 (0.0052)	+0.0004	0.0040	0.96
FoM ordered, factor 2	0.1079 (0.0017)	0.1079 (0.0019)	0.0000	0.0014	1.00
FoM ordered, factor 16	0.4311 (0.0188)	0.4308 (0.0199)	−0.0003	0.0150	0.96
FoM ordered, factor 256	0.6795 (0.0326)	0.6784 (0.0327)	−0.0011	0.0253	1.00

Table 2. Ensemble level, 30 runs each (selection of the 20 metrics; all passed). FoM: figure of merit of the forest-to-built transition at 1999. Tolerance: three standard errors of the difference of means. Differences are computed before rounding.

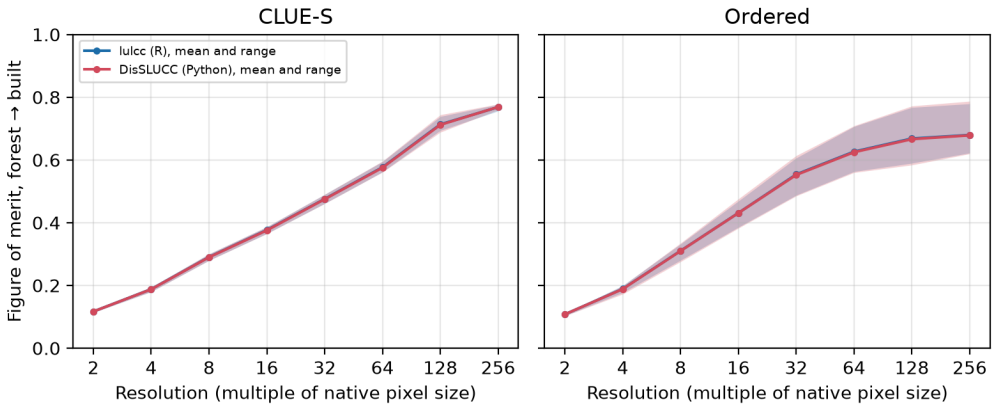


Figure 3. Figure of merit of the forest-to-built transition against resolution, as in Fig. 8 of the original article: mean and range of 30 runs of lulcc and of the replication.

8.3 The published values

The paper prints ten AUCs: those of the three model types for each class (its Fig. 5) and that of the glm for the gain of built (its Fig. 6). Table 3 places them in the spread of our 30 runs of lulcc; the regression trees and random forests are lulcc's own, since the replication does not include them. Nine of the ten values fall inside that spread; the AUC of the glm for forest (0.6423) is slightly below the lowest of the 30 runs (0.6425). The paper used lulcc 1.0 with the R of 2015, our reference lulcc 1.0.4 on current R, and we did not investigate the difference further. It is not unexpected: a value from the same distribution falls outside the range of 30 runs with probability 2/31, about 6%, and at least one of ten does with probability about 49%.

9 Obstacles

The demonstration script does not run unchanged. With current versions of the raster and sp packages, the CLUE-S and ordered allocations stop with `different crs`. The rasters of the data set were serialised by an old version of raster, before the slot that holds the coordinate reference system in WKT form existed; the allocation creates each simulated map from the initial one, the new object carries a CRS in the current form and the old one does not, and the check that precedes stacking them fails. Reassigning the CRS of the old objects does not help: it leaves them with no CRS at all. Rebuilding each

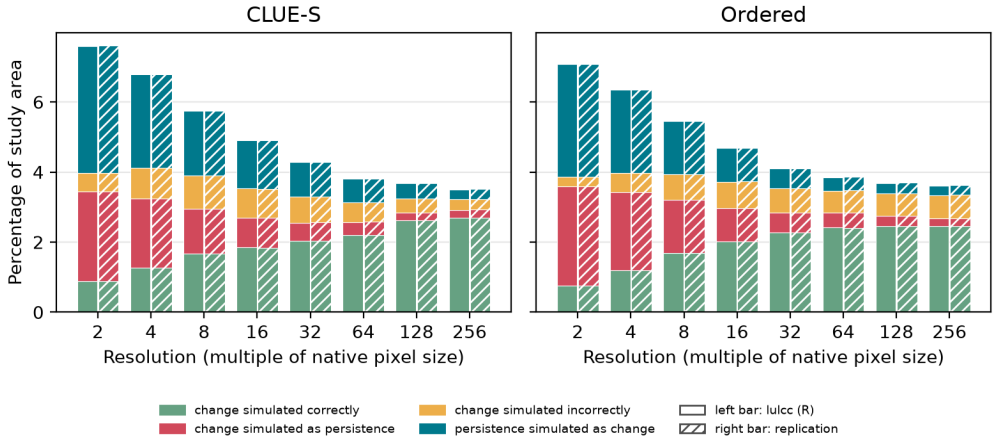


Figure 4. Agreement budget of the forest-to-built transition against resolution, as in Fig. 7 of the original article: mean of 30 runs of lulcc (left bar of each pair) and of the replication (right bar, hatched), in percentage of the study area. Outside the acceptance criteria; see Section 7.

Model	Class	paper	lulcc, 30 runs: min–max	mean	percentile of paper
glm	forest	0.6423	0.6425–0.6452	0.6439	below min
glm	built	0.9384	0.9377–0.9389	0.9382	70
glm	other	0.7276	0.7270–0.7317	0.7295	7
glm	gain of built	0.6448	0.6435–0.6450	0.6446	63
rpart	forest	0.6222	0.6219–0.6362	0.6280	3
rpart	built	0.9158	0.9150–0.9162	0.9156	70
rpart	other	0.6458	0.6357–0.6747	0.6497	60
rf	forest	0.6117	0.6102–0.6178	0.6136	13
rf	built	0.9363	0.9336–0.9371	0.9355	83
rf	other	0.7074	0.7022–0.7103	0.7050	93

Table 3. AUCs printed in the original article (its Figs. 5 and 6, one unseeded run of lulcc 1.0) against 30 runs of lulcc 1.0.4. rpart: regression tree; rf: random forest (in regression mode, Section 4).

layer as a new raster object with the same grid, values and CRS solves it; this is the only change we made to the script. A package written to make land use models reproducible thus needed a repair to run its own published example eleven years later, for a reason unrelated to the model.

An undeclared dependency. lulcc loads the `gsubfn` package when it builds the explanatory variables, but lists it only as a suggestion; it has to be installed by hand.

10 Conclusion

The Plum Island Ecosystems application of lulcc was replicated. With the same inputs, an independent implementation of CLUE-S reproduces lulcc's maps exactly, and so do our ported ordered allocation and validation. That implementation was ported from LuccME and reproduces LuccME's discrete labs, as a port should; what this replication adds is that it also reproduces lulcc, so on this application the two independent readings of the description of CLUE-S, in LuccME and in lulcc, coincide. With independent random numbers, the two implementations agree in distribution on every metric compared. The main obstacle was not in the model but in the environment around it.

Reproducing this replication

Code: https://github.com/LambdaGeo/rescience-lulcc. Reference outputs and the Docker environment that produces them: https://github.com/LambdaGeo/lulcc-goldens. With the reference downloaded, `make env`, then `make check` (about one minute), `make ensemble` (about 15 minutes) and `make compare` reproduce Tables 1 and 2 and Figures 3 and 4; `make maps` draws Figure 2, and `make audit` (about three minutes) runs the checks of Section 8.1. Each takes the reference's directory as `GOLDENS=`. The versions of the Python packages are fixed in the file `requirements.lock`, and the R environment of the reference in the Dockerfile of lulcc-goldens. The code is released under GPL-3.0-or-later, the licence family of lulcc, because part of it follows lulcc's source code.

Declaration of generative AI use

A large language model from Anthropic (Claude Opus 5.5, in the claude.ai interface with code execution) was used throughout this work, from the choice of the article to replicate to this text, starting on 24 October 2026. It searched for and read the original article and lulcc's source code; compared lulcc with DisSLUCC; wrote the scripts that generate the reference outputs, the replication code and its tests, and the comparison scripts; drafted the acceptance criteria; ran a first reference, the deterministic comparison and both ensembles in a cloud workspace; found the cause of the CRS error described in Section 9; and drafted and typeset this article. The decisions were the first author's: replicating this article, generating the reference in a separate archive, using DisSLUCC unchanged and keeping the missing pieces in the replication's repository, comparing at a deterministic and an ensemble level, and keeping 30 runs. The CLUE-S implementation used, DisSLUCC, predates this work. The first author regenerated the reference in the Docker image and reran the comparison on his own computer, with the same results; the authors reviewed the code and the text and are responsible for all the content. The tool is not listed as an author. Commits made with it carry a Co-Authored-By line naming the model.

Acknowledgements

We thank the developers of TerraME and LuccME at INPE, whose open code is the origin of the CLUE-S implementation used here, and S. Moulds for making lulcc and its demonstration openly available.

Complete table of the 20 ensemble metrics: see `results/ensemble_comparison.csv` *in the code repository.*

References

1. S. Moulds, W. Buytaert, and A. Mijic. "An open and extensible framework for spatially explicit land use change modelling: the lulcc R package." In: **Geoscientific Model Development** 8.10 (2015), pp. 3215–3229. DOI: 10.5194/gmd-8-3215-2015.
2. P. H. Verburg, W. Soepboer, A. Veldkamp, R. Limpiada, V. Espaldon, and S. S. A. Mastura. "Modeling the spatial dynamics of regional land use: the CLUE-S model." In: **Environmental Management** 30.3 (2002), pp. 391–405. DOI: 10.1007/s00267-002-2630-x.
3. R. Fuchs, M. Herold, P. H. Verburg, and J. G. P. W. Clevers. "A high-resolution and harmonized model approach for reconstructing and analysing historic land changes in Europe." In: **Biogeosciences** 10 (2013), pp. 1543–1559. DOI: 10.5194/bg-10-1543-2013.
4. R. G. Pontius, S. Peethambaram, and J.-C. Castella. "Comparison of three maps at multiple resolutions: a case study of land change simulation in Cho Don District, Vietnam." In: **Annals of the Association of American Geographers** 101.1 (2011), pp. 45–62. DOI: 10.1080/00045608.2010.517742.

5. N. P. Rougier, K. Hinsen, F. Alexandre, et al. "Sustainable computational science: the ReScience initiative." In: **PeerJ Computer Science** 3 (2017), e142. DOI: 10.7717/peerj-cs.142.
6. National Academies of Sciences, Engineering, and Medicine. **Reproducibility and Replicability in Science**. The National Academies Press, 2019. DOI: 10.17226/25303.
7. A. P. D. Aguiar, T. Carneiro, P. Andrade, and T. Assis. "LuccME-TerraME: an open-source framework for spatially explicit land use change modelling." In: **GLP News** 8 (2012), pp. 21–23.
8. T. G. d. S. Carneiro, P. R. d. Andrade, G. Câmara, A. M. V. Monteiro, and R. R. Pereira. "An extensible toolbox for modeling nature–society interactions." In: **Environmental Modelling & Software** 46 (2013), pp. 104–117. DOI: 10.1016/j.envsoft.2013.03.002.
9. A. P. D. Aguiar et al. "Land use change emission scenarios: anticipating a forest transition process in the Brazilian Amazon." In: **Global Change Biology** 22.5 (2016), pp. 1821–1840. DOI: 10.1111/gcb.13134.
10. F. G. S. Bezerra, C. Von Randow, T. O. Assis, K. R. A. Bezerra, G. Tejada, A. A. Castro, D. M. P. Gomes, R. Avancini, and A. P. D. Aguiar. "New land-use change scenarios for Brazil: refining global SSPs with a regional spatially-explicit allocation model." In: **PLOS ONE** 17.4 (2022), e0256052. DOI: 10.1371/journal.pone.0256052.
11. F.-R. Mazy and P.-Y. Longaretti. "Towards a generic theoretical framework for pattern-based LUCC modeling: allocation revisited: formal foundations and bias identification." In: **Environmental Modelling & Software** 166 (2023), p. 105706. DOI: 10.1016/j.envsoft.2023.105706.
12. S. S. Costa, N. J. Santos Junior, F. M. Sousa, and D. S. Bezerra. **DisSModel: A Python framework for spatially explicit dynamic modeling**. Version 0.6.6. 2026. DOI: 10.5281/zenodo.23246106.
13. S. S. Costa. **DisSLUCC**. Version 0.5.0. 2026. DOI: 10.5281/zenodo.23219338.
14. S. S. Costa. **DisSLUCC benchmark**. Version 0.2.2. 2026. DOI: 10.5281/zenodo.23224097.
15. S. S. Costa, D. d. S. Bezerra, and J. M. P. Alves. **Open LUCC workflow**. Version 1.1.0. 2026. DOI: 10.5281/zenodo.23415814.
16. R. G. Pontius and B. Parmentier. "Recommendations for using the relative operating characteristic (ROC)." In: **Landscape Ecology** 29.3 (2014), pp. 367–382. DOI: 10.1007/s10980-013-9984-8.
17. S. Moulds. **lulcc: Land Use Change Modelling in R**. R package version 1.0.4. 2019. URL: https://cran.r-project.org/package=lulcc.
18. S. S. Costa, D. d. S. Bezerra, J. M. P. Alves, H. B. Oliveira, M. D. Vale, and S. K. S. Zhao. **lulcc-goldens: reference outputs of lulcc 1.0.4 for the Plum Island Ecosystems application**. Version 0.1.0. 2026. URL: https://github.com/LambdaGeo/lulcc-goldens.
19. C. R. Harris, K. J. Millman, S. J. van der Walt, et al. "Array programming with NumPy." In: **Nature** 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.
20. P. Virtanen, R. Gommers, T. E. Oliphant, et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python." In: **Nature Methods** 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.
21. S. Seabold and J. Perktold. "statsmodels: econometric and statistical modeling with Python." In: **Proceedings of the 9th Python in Science Conference**. 2010, pp. 92–96. DOI: 10.25080/Majora-92bf1922-011.